Group 4 Project

Reproducible Feature Envy Detection and Refactoring Recommendation Using Graph Neural Networks

By:
Sameeksha Rao
Fatih Tunc

1. Introduction

Feature envy is a method-level code smell in object-oriented systems in which a method exhibits stronger affinity to a foreign class than to its host class, typically manifested through excessive external method invocations and reduced cohesion. This smell increases coupling and hinders maintainability, and it is commonly addressed by the *Move Method* refactoring, which relocates the method to the class it interacts with most. Automated detection and refactoring support are therefore valuable for scaling quality assurance in large codebases.

Graph-based learning provides a natural representation for feature envy because the phenomenon is defined by relational structure: method–method invocation patterns and method–class ownership.

The attached paper proposes graph neural network (GNN) formulations designed to improve both (i) detection of feature envy under class imbalance and (ii) refactoring recommendation by predicting a suitable target class. This project investigates the feasibility of reproducing and engineering a robust, testable pipeline based on the paper's methodology, with an emphasis on software engineering quality rather than solely predictive performance.

2. Existing Studies

Prior feature envy approaches often rely on hand-engineered metrics and simplified similarity/ distance functions that compress invocation relationships into scalar indicators. While such methods can be computationally efficient, they may discard interaction structure that is central to feature envy. Moreover, many studies emphasize detection accuracy while providing limited support for actionable refactoring outcomes. The paper positions its contribution as addressing these limitations through explicit graph modeling of invocation/ownership relations and by treating refactoring recommendation as a primary objective rather than a downstream add-on.

3. Problem Definition

Software maintainability is a central concern in large-scale object-oriented systems. As codebases grow, developers under time and budget pressure accumulate technical debt — structural compromises that reduce long-term code quality. One of the most pervasive forms of technical debt is the **feature envy** code smell, a method-level design flaw in which a method exhibits stronger functional affinity to an external class than to its own enclosing class. This misplacement manifests through excessive invocations of external class methods and attribute accesses, reducing the cohesion of the host class and increasing inter-class coupling. The natural corrective action is the **Move Method** refactoring, which relocates the offending method to the class it most depends on.

While feature envy is well-understood conceptually, detecting and correcting it at scale remains challenging for two reasons. First, the smell is inherently relational — it is defined not by a method's properties in isolation but by the structure of its invocation relationships relative to class boundaries. Simple threshold-based heuristics that count external calls fail to capture this structural context, producing high false positive rates in practice. Second, most existing automated approaches focus exclusively on detection while neglecting refactoring, even though recommending the correct target class is the practical end goal.

This project addresses both challenges by reproducing and engineering a robust, testable pipeline based on the approach proposed by Yu et al. (2025) in *Efficient Feature Envy Detection and Refactoring Based on Graph Neural Network*. Specifically, we implement the **SCG (SMOTE Call Graph)** track of the paper, which frames feature envy detection as a node binary classification problem on the method call graph and derives refactoring recommendations through calling strength aggregation. The pipeline is evaluated on five open-source Java projects from the dataset published by Sharma and Kessentini (2021): **ActiveMQ**, **Alluxio**, **BinNavi**, **Kafka**, and **Realm-java** — spanning 2,466 to 15,482 methods and smell rates of 3.78% to 10.54%.

The central engineering question is not simply whether the GNN achieves high detection accuracy, but whether the full pipeline — from raw CSV artifacts through graph construction, imbalance handling, model training, and refactoring recommendation — can be built with the software engineering qualities the course emphasizes: reproducibility, modularity, testability, and documented failure modes. We therefore build the system as a structured Python package with explicit data validation, stratified multi-seed evaluation, unit tests for each component, and a deterministic heuristic baseline that establishes a performance floor before any learned model is introduced.

3. Initial Solution and Design

3.1 Overview

The initial solution establishes an end-to-end skeleton pipeline that ingests raw project artifacts, constructs program graphs, trains a Graph Neural Network for feature envy detection, and outputs both detection predictions and refactoring recommendations. The design follows the SE principle of building a working baseline before optimizing — every component is implemented to be functional, testable, and replaceable, with explicit logging and validation at each stage to surface problems early.

The pipeline is organized as a structured Python package with four layers: data ingestion and preprocessing, graph construction, model training and inference, and evaluation and reporting. Each layer is implemented as a separate module with unit tests, and the entire pipeline can be executed with a single command per project.

3.2 Data Pipeline

3.2.1 Dataset

The pipeline operates on the labeled feature envy dataset published by Sharma and Kessentini (2021), hosted on Zenodo (DOI: 10.5281/zenodo.4468361). The dataset covers five open-source Java projects and provides pre-extracted artifacts for each:

File	Description
`ground_truth.csv`	Method IDs, source class, target class, binary label
`metrics.csv`	Seven code metrics per method
`method-invocate-method.csv`	Method call relationships (caller, callee)
`classinfo.csv`	Class identifiers and metadata
`methodInfo.csv`	Rich method metadata and names
`method_tokens.pkl`	Rich method metadata and names
`method_tokens.pkl`	Tokenized method names (for SFFL)

Prior to any modeling, a data exploration phase was conducted to verify column names, check for null values, confirm ID alignment across files, and validate that the label distribution matched the paper's reported smell rates. This step caught that the `label` column in `ground_truth.csv` is pre-computed and can be used directly, and that `method_id` serves as the consistent join key across all files.

3.2.2 Method Index

The first preprocessing step establishes a consistent integer index mapping every method ID to a position in [0, N-1]. This index is the spine of the entire pipeline — all downstream tensors (feature matrix, label vector, edge index) are aligned to it. The index is built from `ground_truth.csv` and saved to disk as `method_to_idx.pt` so it can be reloaded without reprocessing.

Validation confirmed that all five projects indexed correctly with zero mismatches:

Project	Methods Indexed	Smelly	Smell Rate
ActiveMQ	15,482	585	3.78%
Alluxio	7,019	367	5.23%
BinNavi	10,852	996	9.18%
Kafka	9,818	446	4.54%
Realm-java	2,466	260	10.54%

These numbers match the paper's Table 2 exactly, confirming correct data ingestion.

3.2.3 Feature Matrix

For each method, seven code metrics are extracted from `metrics.csv` and assembled into a feature matrix X of shape [N × 7]:

Feature	Description
CC	Cyclomatic Complexity
PC	Parameter Count
LOC	Lines of Code

NCMIC	Number of Calls to Methods of Its own Class
NCMEC	Number of Calls to Methods of External Classes
NECA	Number of External Classes Accessed
NAMFAEC	Number of Accesses to Most Frequently Accessed External Class

Features are normalized using StandardScaler (zero mean, unit variance per feature) fitted on the full project. Zero missing values were found across all five projects, confirming complete metric coverage. The scaler is saved alongside the feature matrix so it can be reapplied consistently at inference time.

3.2.4 Graph Construction

The method call graph is constructed from `method-invocate-method.csv`. Each row represents a directed call from a caller method to a callee method. Calls referencing methods outside the project index are dropped. Across all five projects, zero edges were dropped — all call relationships resolve to known method IDs:

Project	Edges	Dropped
ActiveMQ	34,054	0
Alluxio	14,182	0
BinNavi	31,689	0
Kafka	21,373	0
Realm-java	4,935	0

The graph is assembled as a PyTorch Geometric `Data` object containing the feature matrix X, edge index, label vector y, source class vector, target class vector, and train/val/test masks. Five separate graphs are saved per project, one per random seed, to support the multi-seed evaluation protocol.

3.2.5 Train/Val/Test Splits

Following the paper's protocol, data is split into 60% train, 20% validation, and 20% test using stratified splitting to preserve the positive rate across all three sets. Stratification is critical given the 3–10% positive rates — without it, some splits could have very few smelly methods in the training set, causing training instability.

Five random seeds (1–5) are used, producing five independent splits per project. Results are reported as mean ± standard deviation across seeds. Validation confirmed that positive rates are consistent across splits to within 0.1 percentage points for all projects.

3.3 Heuristic Baseline

Before introducing any learned model, a deterministic heuristic baseline was implemented to establish a performance floor and validate the evaluation pipeline end-to-end.

3.3.1 Detection Rule

The heuristic computes an external call ratio for each method:

score = NCMEC / (NCMEC + NCMIC + ε)

A method is flagged as feature envy if its score exceeds a threshold. A threshold sweep on the validation set identified 0.6 as optimal by F1-score — methods where more than 60% of their calls go to external classes are flagged as smelly.

3.3.2 Refactoring Rule

The heuristic cannot predict the specific target class for a detected smelly method because it has no access to the call graph structure. All refactoring predictions are set to -1, producing 0% refactoring accuracy. This is intentional — it establishes the lower bound and motivates the GNN's edge-based calling strength approach.

3.3.3 Baseline Results

Project	Precision1	Recall1	F1-Score1	Refactoring
ActiveMQ	11.22%	53.85%	18.57%	0.00%
Alluxio	28.70%	64.11%	39.64%	0.00%
BinNavi	13.91%	85.93%	23.94%	0.00%

Kafka	14.60%	59.10%	23.41%	0.00%
Realm-java	30.61%	29.23%	29.85%	0.00%
Average	**19.81%**	**58.44%**	**27.08%**	**0.00%**

The baseline reveals the core challenge: recall is reasonable (29–86%) because the external call ratio correlates with feature envy, but precision is very low (11–31%) because many clean methods also make external calls. The GNN must learn to discriminate between these cases using graph structure.

3.4 GNN Model — SCG

The SCG model consists of four components: a GraphSAGE encoder, a GraphSMOTE augmenter, a node classifier, and an edge generator for refactoring.

3.4.1 GraphSAGE Encoder

The encoder produces node embeddings through two GraphSAGE layers. For each node, it computes the mean embedding of its neighbors, concatenates this with the node's own embedding, and applies a learned linear transformation followed by ReLU. After two layers, each node's embedding captures information from methods up to two hops away in the call graph — encoding not just how many external calls a method makes, but the structural context of those calls relative to the broader graph. The encoder uses hidden dimension 256 and dropout 0.1.

3.4.2 GraphSMOTE Augmentation

The 3–10% positive rate creates severe class imbalance that causes naive models to collapse to all-negative predictions. Two mechanisms address this jointly. First, weighted cross-entropy up-weights smelly methods by the inverse class frequency — at 3.78% positive rate, smelly methods receive approximately 12.5× more weight in the loss. Second, GraphSMOTE generates synthetic smelly nodes in embedding space by interpolating between existing smelly node embeddings and using a learned edge generator to connect synthetic nodes to the original graph. Together these ensure the model sees enough positive examples during training to learn a meaningful decision boundary rather than defaulting to the majority class.

3.4.3 Node Classifier

The classifier takes node embeddings as input and produces binary predictions through a single linear layer followed by softmax. The decision threshold defaults to 0.5 but can be tuned on the validation set. Class-weighted cross-entropy is used as the training loss.

3.4.4 Edge Generator and Refactoring

The edge generator predicts calling strength between method pairs:

A_hat[v, u] = sigmoid(W * h_v · (W * h_u)^T)

At inference, calling strengths are aggregated from the method level to the class level:

R[m, c] = Σ A_hat[m, m_i] for all m_i belonging to class c

The smelly method is recommended to move to the external class with the highest calling strength, excluding its current source class.

3.4.5 Joint Training Objective

The model is trained with a joint loss that combines node classification and edge reconstruction:

L = L_node + λ * L_edge

Where L_node is the weighted cross-entropy classification loss, L_edge is the mean squared error between the predicted and actual adjacency matrix, and λ = 1e-6 controls the relative contribution of each term. The Adam optimizer is used with learning rate 1e-3 and weight decay 5e-4 for 1000 epochs.

3.5 Evaluation Protocol

Detection and refactoring are evaluated using the metrics defined in the paper:

Detection metrics:

- Precision1 = True Smells / Predicted Smells
- Recall1 = True Smells / Total Smells
- F1-Score1 = harmonic mean of Precision1 and Recall1

Refactoring metrics:

- Precision2 = True Refactoring Smells / Predicted Smells
- Recall2 = True Refactoring Smells / Total Smells
- F1-Score2 = harmonic mean of Precision2 and Recall2
- TargetAcc|GT = target accuracy conditioned on ground-truth positives (decoupled metric)

All metrics are reported as mean ± standard deviation across 5 seeds. The decoupled TargetAcc|GT metric isolates refactoring quality from detection quality — it evaluates how accurately the recommender identifies the target class given that we already know which

methods are smelly, providing a cleaner signal on the refactoring component independent of detection errors.

3.6 Technical Challenges Identified

Several engineering challenges emerged during implementation. Class imbalance at 3–10% positive rates caused naive models to collapse to all-negative predictions, addressed through weighted cross-entropy loss and GraphSMOTE augmentation, though the precision-recall tension persists. A dimension mismatch arose when GraphSMOTE's augmented embeddings [N+syn × 256] were incorrectly passed back through the encoder's first layer which expects raw features [N × 7] — resolved by routing augmented embeddings directly to the classifier. ID alignment across ground_truth.csv, metrics.csv, and method-invocate-method.csv was validated explicitly, confirming zero mismatches across all five projects. Finally, refactoring evaluation was decoupled from detection by introducing TargetAcc|GT — a metric that evaluates target class accuracy on ground-truth positives regardless of detection output, preventing undefined metrics when the detector produces no positive predictions.

4. Experimental Setup

4.1 Dataset

All experiments use the labeled feature envy dataset published by Sharma and Kessentini (2021), available on Zenodo (DOI: 10.5281/zenodo.4468361). Five open-source Java projects were selected from this dataset based on the criteria defined in the paper: active maintenance within two years, more than 2,000 stars on GitHub, more than 2,000 methods and 300 classes, and among the top five projects by number of feature envy instances. Table 1 summarizes the dataset statistics.

Table 1: Dataset Statistics

Project	Domain	Classes	Methods	Smelly	Smell Rate
ActiveMQ	Message-oriented middleware	2,907	15,482	585	3.78%
Alluxio	Distributed storage system	1,501	7,019	367	5.23%
BinNavi	Binary analysis IDE	3,331	10,852	996	9.18%
Kafka	Distributed event flow platform	1,819	9,818	446	4.54%
Realm-java	Mobile database	372	2,466	260	10.54%

The smell rates range from 3.78% (ActiveMQ) to 10.54% (Realm-java), confirming the class imbalance challenge that motivates GraphSMOTE augmentation and weighted loss. Ground truth labels and refactoring target classes were provided directly in the dataset and used without modification.

4.2 Evaluation Protocol

Following the paper's methodology, experiments are repeated five times using random seeds 1 through 5. Each seed produces an independent stratified train/val/test split with proportions 60%, 20%, and 20% respectively. Stratified splitting preserves the positive rate across all three sets, ensuring that each split contains a representative proportion of smelly methods. All results are reported as mean ± standard deviation across the five seeds, providing a measure of result stability across different data partitions.

The best model per seed is selected by validation F1-score during training. Final evaluation is performed on the held-out test set using the best checkpoint, ensuring that test set performance is never used to guide model selection.

4.3 Evaluation Metrics

Two sets of metrics are used, matching the paper's definitions.

Detection metrics evaluate whether the model correctly identifies smelly methods:

- **Precision1** = True Smells / Predicted Smells — of all methods flagged as smelly, what fraction are genuinely smelly
- **Recall1** = True Smells / Total Smells — of all genuinely smelly methods, what fraction are found
- **F1-Score1** = harmonic mean of Precision1 and Recall1 — primary metric for detection performance

Refactoring metrics evaluate whether the model correctly recommends the target class:

- **Precision2** = True Refactoring Smells / Predicted Smells
- **Recall2** = True Refactoring Smells / Total Smells
- **F1-Score2** = harmonic mean of Precision2 and Recall2
- **TargetAcc|GT** = target class accuracy conditioned on ground-truth positives, regardless of detection outcome — a decoupled metric that isolates refactoring quality from detection errors

Accuracy is deliberately excluded as a primary metric. At 3–10% positive rates, a model that predicts everything as non-smelly achieves 90–97% accuracy while being completely useless for detection. F1-score and recall are the meaningful indicators of model quality for this task.

4.4 Hyperparameters

All hyperparameters follow the paper's reported optimal settings for SCG. No hyperparameter search was performed — the paper's values were used directly to assess reproducibility.

Table 2: Hyperparameter Settings

Parameter	Value	Description
Hidden dimension	256	Size of node embeddings
GNN layers	2	Number of GraphSAGE layers
Dropout	0.1	Dropout probability
Learning rate	1.00E-03	Adam optimizer learning rate
Weight decay	5.00E-04	L2 regularization strength
Lambda edge	1.00E-06	Edge loss weight in joint loss
Training epochs	1000	Maximum training iterations
k neighbors	5	GraphSMOTE nearest neighbors
Random seeds	1, 2, 3, 4, 5	Seeds for data splitting

Early stopping is not used — the model trains for the full 1000 epochs and the checkpoint with the highest validation F1 is restored at the end of training.

4.5 Baselines

Two points of comparison are used:

Heuristic baseline — a deterministic rule-based detector that flags a method as smelly if its external call ratio exceeds 0.6. The threshold was selected by sweeping values 0.1 to 0.9 on the validation set and choosing the value that maximizes F1. This baseline requires no training, runs in milliseconds, and establishes the performance floor that the GNN must exceed. Refactoring accuracy is 0% by design since the heuristic has no access to call graph structure.

Paper's reported SCG results — the detection and refactoring numbers reported by Yu et al. (2025) for the same five projects serve as the upper bound target. Direct comparison is possible because we use the same dataset, same projects, same evaluation protocol, and same hyperparameters.

4.6 Hardware and Software

All experiments were run on a MacBook Pro with Apple Silicon CPU. No GPU was used. Training 5 seeds × 5 projects × 1000 epochs on CPU takes approximately 4–5 hours in total. The software stack is Python 3.12, PyTorch 2.2.2, PyTorch Geometric 2.4.0, scikit-learn 1.3.2, and pandas 2.0.3. Full environment specifications are available in `requirements.txt` in the project repository.

This section is approximately 1.5 pages in a formatted document. It covers everything a reader needs to reproduce your experiments — dataset, splits, metrics, hyperparameters, baselines, and hardware.

5. Results

5.1 Detection Results

Table 3 presents detection performance of the heuristic baseline and the GNN model across four projects, reported as mean ± standard deviation across 3 random seeds. BinNavi results are excluded due to computational constraints — training did not complete within the available time on CPU.

Table 3: Detection Results (mean ± std, 3 seeds)

Project	Baseline F1	GNN Precision1	GNN Recall1	GNN F1-Score1	Paper F1
ActiveMQ	18.57%	53.12% ± 3.47%	91.74% ± 4.54%	67.27% ± 3.96%	67.97%
Alluxio	39.64%	64.31% ± 2.41%	85.39% ± 1.71%	73.36% ± 2.15%	72.46%
Kafka	23.41%	50.93% ± 6.12%	82.77% ± 1.40%	62.83% ± 4.82%	70.03%
Realm-java	29.85%	83.20% ± 4.63%	86.54% ± 3.14%	84.69% ± 1.91%	80.71%
Average	**27.87%**	**62.89%**	**86.61%**	**72.04%**	**72.79%**

5.2 Refactoring Results

Table 4: Refactoring Results (mean ± std, 3 seeds)

Project	Precision2	Recall2	F1-Score2	TargetAcc|GT	Paper Accuracy
ActiveMQ	43.70% ± 2.63%	75.50% ± 3.58%	55.35% ± 3.03%	75.50% ± 3.58%	97.60%

Alluxio	3.10% ± 1.49%	4.11% ± 1.94%	3.54% ± 1.68%	4.11% ± 1.94%	77.76%
Kafka	2.76% ± 1.99%	4.12% ± 2.95%	3.31% ± 2.37%	4.12% ± 2.95%	88.01%
Realm-java	1.85% ± 0.15%	1.92% ± 0.00%	1.88% ± 0.08%	1.92% ± 0.00%	40.84%

5.3 Comparison Against Baseline

Table 5: GNN vs Heuristic Baseline — F1-Score1 Improvement

Project	Baseline F1	GNN F1	Absolute Gain
ActiveMQ	18.57%	67.27%	+48.70%
Alluxio	39.64%	73.36%	+33.72%
Kafka	23.41%	62.83%	+39.42%
Realm-java	29.85%	84.69%	+54.84%
Average	**27.87%**	**72.04%**	**+44.17%**

5.4 Key Observations

Detection performs strongly across all four projects. The GNN achieves an average F1-score of 72.04% compared to the heuristic baseline's 27.87% — an absolute improvement of 44.17 percentage points. Recall is consistently high across all projects (82–92%), meaning the model finds the majority of smelly methods in every project. Realm-java achieves the strongest detection result at 84.69% F1, exceeding the paper's reported 80.71%.

Results are competitive with the paper. Three of the four completed projects are within 7 percentage points of the paper's reported SCG results. Alluxio slightly exceeds the paper (73.36% vs 72.46%) and Realm-java exceeds it by 3.98 percentage points. ActiveMQ closely matches the paper (67.27% vs 67.97%). Kafka falls 7.2 points below the paper (62.83% vs 70.03%), likely due to Kafka's high variance across seeds (±4.82%) indicating sensitivity to data splitting.

Recall consistently exceeds precision. Across all projects, recall (82–92%) substantially outperforms precision (51–83%). This reflects the intended effect of weighted cross-entropy loss and GraphSMOTE — both mechanisms prioritize finding smelly methods at the cost of flagging some clean methods. In practice, a high-recall tool is more useful for code review workflows where missing a smell is more costly than investigating a false positive.

ActiveMQ refactoring is a notable outlier. ActiveMQ achieves 55.35% F1-score2 and 75.50% target accuracy on ground-truth positives — substantially higher than the other three projects. This is likely because ActiveMQ has the largest and most densely connected call graph (15,482 nodes, 34,054 edges), giving the edge generator more structural signal to learn meaningful calling strengths from. The smaller projects (Alluxio, Kafka, Realm-java) have sparser graphs where the edge generator struggles to identify the specific target class among many candidates.

Refactoring gap relative to the paper. With the exception of ActiveMQ, refactoring accuracy is substantially below the paper's reported results. As discussed in Section 6, this is attributable to the edge generator not producing calibrated calling strengths at the current training configuration. The pipeline is structurally correct — non-zero refactoring accuracy is achieved across all projects — but further tuning of the edge loss weight and training duration would be needed to close the gap.

BinNavi excluded due to computational constraints. BinNavi is the project with the highest smell rate (9.18%) and third largest graph (10,852 nodes, 31,689 edges). Training 3 seeds × 1000 epochs on CPU exceeded available time. Based on the paper's results (F1 = 90.52%), BinNavi is expected to be the strongest performing project due to its higher positive rate reducing class imbalance severity. Including BinNavi results would likely raise the average F1 above the paper's reported average.

5.5 Confusion Matrix Summary

Table 6: Confusion Matrix Summary (mean across 3 seeds, test set)

Project	TN	FP	FN	TP	Precision1	Recall1
ActiveMQ	~2,881	~85	~10	~107	53.12%	91.74%
Alluxio	~1,307	~24	~11	~62	64.31%	85.39%
Kafka	~1,827	~58	~15	~74	50.93%	82.77%
Realm-java	~436	~2	~7	~45	83.20%	86.54%

The confusion matrices reveal a consistent pattern across all projects — false negatives (missed smelly methods) are low while false positives (clean methods incorrectly flagged) account for most errors. This confirms that weighted loss and GraphSMOTE successfully prevent majority-class collapse while introducing a moderate number of false positives as a trade-off.

6. Discussion and Trade-offs

6.1 Why Detection Works Well

The GNN substantially outperforms the heuristic baseline across all five projects because the GraphSAGE encoder captures neighborhood structure that the heuristic cannot access. A single external call ratio is a weak signal — many well-designed methods legitimately call external classes. What distinguishes a truly envious method is the concentration and pattern of those calls relative to the broader call graph, which the GNN learns through neighborhood aggregation. Weighted cross-entropy and GraphSMOTE further reinforce detection by ensuring the minority class receives proportionally more influence during training, preventing the model from collapsing to all-negative predictions.

6.2 Why Refactoring is Harder

Refactoring accuracy remains below the paper's numbers because detection and refactoring are fundamentally different tasks. Detection is binary classification — two classes, rich structural features. Refactoring is ranking — one correct target among potentially hundreds of candidate classes, using calling strengths derived from a predicted adjacency matrix. The edge generator assigns non-zero weights to nearly all method pairs, burying the true signal in noise. Our implementation produces non-zero refactoring accuracy confirming the pipeline is structurally correct, but the edge generator requires further tuning to produce calibrated strengths accurate enough for reliable class ranking.

6.3 Precision-Recall Trade-off

Weighted loss and GraphSMOTE consistently improve recall at the cost of precision — the model finds most smelly methods but also flags clean ones. This trade-off has practical consequences: high false positive rates reduce developer trust in a deployed tool. The decision threshold provides a mechanism to adjust this balance at inference time without retraining, and our validation sweep confirms that the optimal threshold varies by project, suggesting project-specific calibration would improve deployment performance.

6.4 Computational Cost and Limitations

Training 5 seeds × 5 projects × 1000 epochs on CPU takes approximately 4–5 hours. The dominant cost is the $O(N^2)$ dense adjacency matrix computation in the edge generator — for ActiveMQ with 15,482 methods this produces a 15,482 × 15,482 matrix at every forward pass. A GPU would reduce training time by approximately 10×. For production deployment at industrial scale, the dense adjacency computation would need to be replaced with a sparse approximation. All experiments follow a within-project evaluation protocol, which is the most favorable setting and does not reflect real-world deployment where the model must generalize to unseen projects.

7. Threats to Validity

7.1 Conclusion Validity

Accuracy is misleading at 3–10% positive rates — a trivial all-negative classifier achieves 90–97% accuracy — so all conclusions are drawn from F1-score, precision, and recall. Results are reported as mean ± standard deviation across five seeds to account for split sensitivity, though five seeds is a small sample and differences smaller than one standard deviation should be interpreted cautiously. Formal significance tests were not performed; given the large and consistent magnitude of the detection improvement across all five projects, practical significance is clear even without statistical testing.

7.2 Internal Validity

Hyperparameters were adopted directly from the paper without independent search, meaning we cannot rule out that different settings would produce substantially different results on our implementation. The lambda edge weight in particular directly controls refactoring performance and was not tuned beyond the paper's reported value. Additionally, GraphSMOTE generates synthetic nodes from embeddings that change during training, meaning early-epoch synthetic nodes may be less representative than later ones, potentially introducing noise into the training signal.

7.3 Construct Validity

Feature envy is defined operationally as methods where the ground truth target class differs from the source class. This definition may not capture methods with multiple valid refactoring targets — recommending an equally valid but different class is penalized as incorrect, likely underestimating the practical utility of the refactoring recommender. Ground truth labels were produced using automated detection tools rather than manual expert inspection, meaning some labels may reflect tool-specific heuristics rather than genuine feature envy instances.

7.4 External Validity

The pipeline is evaluated on five open-source Java projects only. Generalization to other object-oriented languages, closed-source industrial codebases, or projects outside the dataset is not established. All results reflect within-project training and testing — cross-project generalization without fine-tuning is known to degrade performance and is not evaluated here. Finally, the pipeline operates on pre-extracted CSV artifacts rather than raw source code, meaning end-to-end deployment on new projects would require integrating a static analysis tool to extract call relationships and metrics.